چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۲

لایهٔ شبکه — بخش ۲: TLS، پین کردن گواهی، OAuth2 و API در مقیاس بالا



اسماعیل مرادی

@esmaeeil-moradi700

چرا امنیت شبکه در ۲۰۲۶ مهم است؟

HTTPS حالا پیش‌فرض است، پس سؤال‌های مصاحبه یک سطح بالاتر رفته‌اند: *روی TLS چه می‌سازید، و وقتی می‌سازید چه چیزی ممکن است خراب شود؟* فین‌تک، سلامت و هر اپی که حساب کاربری دارد این را می‌پرسد:

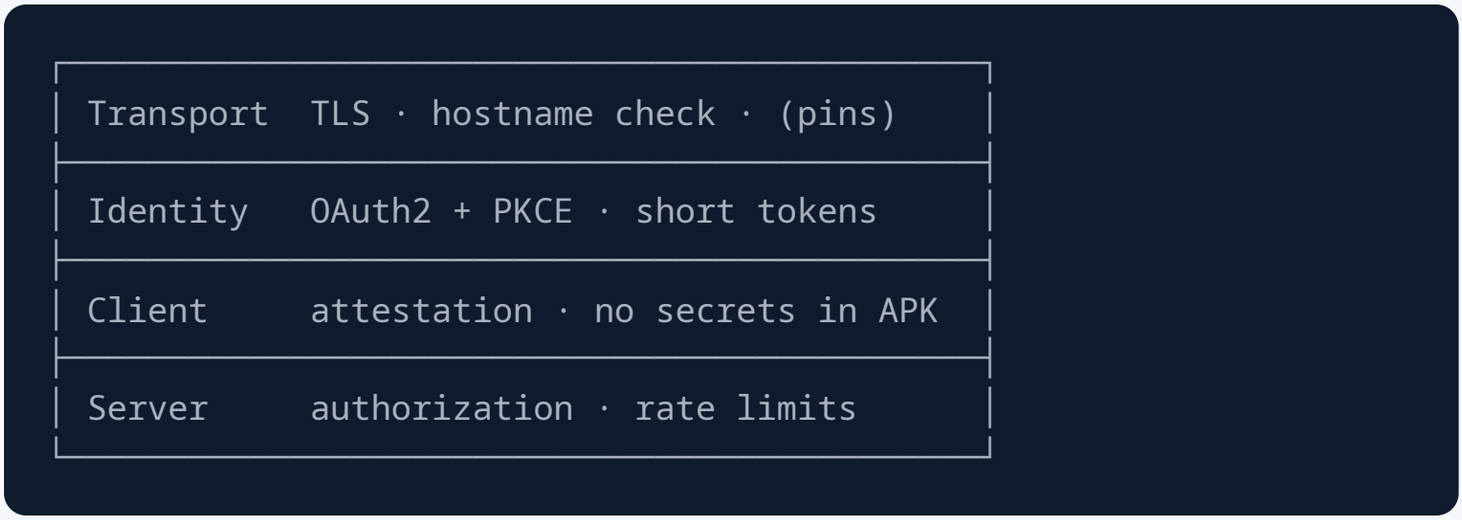
- TLS و **Network Security Config** و اینکه از چه چیزی محافظت می‌کنند — و از چه چیزی نه
- **پین کردن گواهی (certificate pinning)** یک مصالحه با ریسک واقعی قطعی سرویس است، نه یک تیک در چک‌لیست
- **OAuth2 با PKCE** و refresh توکن در شرایط هم‌روندی — یک سؤال کدنویسی کلاسیک سینیور
- secretها: چرا هیچ چیزی داخل APK مخفی نیست
- مقیاس: pagination، محدودیت نرخ (rate limit)، GraphQL، gRPC و WebSocket

سطح	سؤال‌ها
جونیور	TLS و HTTPS، Network Security Config، روش‌های احراز هویت
مید	پین کردن گواهی، ریسک‌های پین، OAuth2 با PKCE، توکن‌ها و Authenticator
سینیور	refresh هم‌زمان، کلید API و secretها، pagination و rate limit
آرکیتکت	فراتر از REST، اتصال‌های real-time، معماری امنیت



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: لایه‌های دفاع



- **TLS** ثابت می‌کند با سرور درست حرف می‌زنید — دربارهٔ اینکه *کلاینت* کیست هیچ چیزی نمی‌گوید
- **توکن‌ها** کاربر را معرفی می‌کنند؛ عمرشان را کوتاه و قابل refresh نگه دارید
- **APK عمومی است:** تنها جایی که قواعد اعمال می‌شوند سرور است

دفاع در عمق

هر لایه فرض می‌کند لایهٔ بالایی‌اش ممکن است شکست بخورد. جواب خوب اول تهدید را نام می‌برد، بعد کنترل را.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

TLS و HTTPS، Network Security Config، روش‌های احراز هویت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. TLS چه چیزی به شما می‌دهد



- **محرمانگی** (رمزنگاری)، **یکپارچگی** (دست‌کاری شناسایی می‌شود)، **احراز هویت سرور** (زنجیرهٔ گواهی و تطبیق hostname)
- اندروید ۱۰ به بعد TLS 1.3 را به‌طور پیش‌فرض فعال می‌کند؛ اپ‌هایی که API 28 به بالا را هدف می‌گیرند به‌طور پیش‌فرض HTTP بدون رمز (cleartext) را مسدود می‌کنند

تلهٔ مصاحبه

«*HTTPS همه چیز را مخفی می‌کند.*» — hostname دیده می‌شود (از طریق SNI و DNS). مسیر، query، هدرها و body رمز می‌شوند — ولی توکنی که در query string باشد باز هم سر از لاگ سرور، proxyها و ابزارهای آنالیتیکس درمی‌آورد. توکن را در هدر بفرستید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. اشتباه‌هایی که TLS را از کار می‌اندازند

تلهٔ مصاحبه

«حل کردن» خطای *SSL handshake failed* با یک `X509TrustManager` که به همه اعتماد می‌کند، یا یک `HostnameVerifier` که همیشه `true` برمی‌گرداند. این کار احراز هویت سرور را برای همهٔ فراخوانی‌ها خاموش می‌کند — و Google Play پیاده‌سازی‌های ناامن را علامت می‌زند. زنجیرهٔ سرور را درست کنید (معمولاً یک intermediate جا افتاده) یا برای CA محیط توسعه از Network Security Config استفاده کنید.

سؤال تکمیلی

«*آیا کاربر می‌تواند ترافیک HTTPS اپ شما را ببیند؟*» — روی دستگاه خودش، بله: با root، یا با buildی که به CAهای کاربر اعتماد می‌کند، یک proxy مثل mitmproxy ترافیک را باز می‌کند. TLS کاربر را **از دیگران** محافظت می‌کند؛ API شما را از کاربرانش پنهان نمی‌کند.

قاعدهٔ سرانگشتی

فرض کنید هر درخواستی را کسی که گوشی را در دست دارد می‌تواند بخواند. API را طوری طراحی کنید که این امن باشد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. Network Security Config

```
<!-- AndroidManifest.xml -->
<application
    android:networkSecurityConfig="@xml/network_security_config">

<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false" />
    <debug-overrides>
        <trust-anchors>
            <certificates src="user" />
        </trust-anchors>
    </debug-overrides>
</network-security-config>
```

- اعلانی و برای کل اپ: cleartext برای هر دامنه، trust anchorها، پین‌ها — بدون کد
- اپ‌هایی که API 24 به بالا را هدف می‌گیرند **به CAهایی که کاربر نصب کرده اعتماد نمی‌کنند**؛ `debug-overrides` فقط روی buildهای debuggable اعمال می‌شود
- به سرور توسعهٔ محلی cleartext لازم دارید؟ یک `domain-config` در نسخهٔ این فایل در source set مربوط به **debug** اضافه کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*در release به CAهای کاربر اعتماد کن تا تیم QA بتواند از Charles استفاده کند.*» — آن‌وقت هر CA که کاربر نصب کرده — یک proxy سازمانی، یک پروفایل مخرب — می‌تواند ترافیک کاربران شما را بخواند. `debug-overrides` دقیقاً برای همین است.

تلهٔ مصاحبه

گذاشتن `cleartextTrafficPermitted="true"` در `base-config` چون یک CDN تصویر هنوز HTTP است. این کار cleartext را برای **همهٔ** hostها باز می‌کند. آن را با یک `domain-config` به همان host محدود کنید — بهتر از آن، CDN را به HTTPS ببرید.

سؤال تکمیلی

«*آیا OkHttp به آن احترام می‌گذارد؟*» — بله. OkHttp از trust manager پلتفرم استفاده می‌کند و cleartext به hostهایی که policy منع کرده را رد می‌کند و خطای `UnknownServiceException` می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. مقایسهٔ روش‌های احراز هویت

روش	چه چیزی فرستاده می‌شود	چه کسی را معرفی می‌کند	نکته
Basic	`base64(user:password)` در هر فراخوانی	کاربر	کدگذاری است، نه رمزنگاری؛ رمز عبور مدام در رفت‌وآمد است
کلید API	یک کلید ثابت در هدر	اپ یا پروژه	بعد از انتشار عمومی است (سؤال ۹)
کوکی session	کوکی که سرور تنظیم می‌کند	یک session	مخصوص مرورگر؛ محافظت CSRF لازم دارد
توکن Bearer	access token کوتاه‌عمر	کاربر و scopeها	همراه refresh token — پیش‌فرض موبایل

تلهٔ مصاحبه

«*JWT رمزنگاری شده است.*» — یک JWT معمولی فقط **امضا** شده است: هر کسی می‌تواند payload آن را با Base64 باز کند. هرگز secret در آن نگذارید، و هرگز نگذارید اپ بر اساس آن تصمیم مجوزدهی بگیرد — امضا را سرور بررسی می‌کند.

سؤال تکمیلی

«*اصلاً اپ باید JWT را بخواند؟*» — خواندن `exp` برای اینکه کمی زودتر refresh کنید اشکالی ندارد. اعتماد به claimهایش اشکال دارد.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

پین کردن گواهی، ریسک‌های پین، OAuth2 با PKCE، توکن‌ها و Authenticator



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. پین کردن گواهی با OkHttp

```
val pinner = CertificatePinner.Builder()
    .add(
        "api.example.com",
        // current key
        "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=",
        // backup key, kept offline and not yet deployed
        "sha256/BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=",
    )
    .build()

val client = OkHttpClient.Builder()
    .certificatePinner(pinner)
    .build()
```

- پین، هش SHA-256 از **کلید عمومی** گواهی (SPKI) است — تمدید گواهی با همان کلید، پین را معتبر نگه می‌دارد
- اتصال وقتی قبول می‌شود که **هر کدام** از گواهی‌های زنجیرهٔ تأییدشده با یک پین جور باشد
- پین **علاوه بر** اعتبارسنجی عادی TLS چک می‌شود، هرگز به‌جای آن



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. گرفتن پین — و اینکه از چه چیزی محافظت می‌کند

```
openssl s_client -connect api.example.com:443 \
    -servername api.example.com </dev/null \
  | openssl x509 -pubkey -noout \
  | openssl pkey -pubin -outform der \
  | openssl dgst -sha256 -binary \
  | openssl enc -base64
```

تلهٔ مصاحبه

«*پین کردن جلوی گواهی‌های جعلی از CAهای نامعتبر را می‌گیرد.*» — روایت پارسال. TLS عادی همین حالا آن‌ها را رد می‌کند. پین از شما در برابر یک CA **معتبر** محافظت می‌کند که برای دامنهٔ شما گواهی صادر کند — CAی هک‌شده، تحت فشار، یا CAی که به مخزن سیستمی دستگاه اضافه شده.

تلهٔ مصاحبه

«*پین کردن جلوی مهندسی معکوس API ما را می‌گیرد.*» — روی دستگاهی که در اختیار مهاجم است، ابزارهای hook در چند دقیقه پین را غیرفعال می‌کنند. پین **کاربران** را از مهاجم شبکه محافظت می‌کند، نه API شما را از کاربرانش.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. ریسک‌های پین: چه چیزی را پین کنیم

هدف پین	میزان محافظت	ریسک چرخش کلید
کلید leaf خودتان	بیشترین	زیاد — هر تغییر کلید اول یک نسخهٔ جدید اپ لازم دارد
کلید CA میانی	خوب	متوسط — اگر CA زنجیرهٔ میانی را عوض کند می‌شکند
کلید CA ریشه	ضعیف‌تر	کمتر — اگر CA را عوض کنید می‌شکند
کلید پشتیبان (آفلاین، استفاده‌نشده)	چرخش برنامه‌ریزی‌شده را ممکن می‌کند	همیشه یکی داشته باشید

- نسخه‌های قدیمی اپ پین‌هایشان را **برای همیشه** نگه می‌دارند — یک چرخش اشتباه، همهٔ نصب‌هایی را که به‌روز نشده‌اند از کار می‌اندازد
- ویژگی `<pin-set expiration="…">` در Network Security Config: بعد از آن تاریخ، پین به‌جای شکست **خاموش می‌شود** — یک سوپاپ اطمینان داخلی
- hostهایی را که در اختیارتان نیست پین نکنید (CDN، آنالیتیکس، SDKهای شخص ثالث) — بدون اجازهٔ شما کلیدشان را عوض می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. اصلاً پین کنیم؟

تلهٔ مصاحبه

«*مشکل چرخش را با پین‌های پویا که از یک endpoint پین‌شده دانلود می‌شوند حل کن.*» — پیشنهاد پارسال. پین اولیه هنوز در کد است، و حالا یک سرویس توزیع پین دارید که هرگز نباید اشتباه کند. ریسک را جابه‌جا می‌کند، حذف نمی‌کند.

سؤال تکمیلی

«*شما پین می‌کنید؟*» — خود مستندات اندروید هشدار می‌دهد که پین برای بیشتر اپ‌ها پرریسک است. من فقط هدف‌های باارزش (بانک، کیف پول) را وقتی پین می‌کنم که کلیدها دست خودمان است: پین پشتیبان، تاریخ انقضا، telemetry برای شکست‌ها و یک runbook برای چرخش. در غیر این صورت: TLS محکم به‌علاوهٔ پایش Certificate Transparency که وقتی هر CAی برای دامنهٔ شما گواهی صادر کند خبرتان می‌کند.

نظر من

پین کردن یک تعهد عملیاتی است، نه قابلیت امنیتی‌ای که روشنش کنید. اگر کسی مالک تقویم چرخش کلید نیست، پین نکنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. OAuth2 با PKCE برای اپ‌های native

```
1 App      makes code_verifier (random, stays in app)
           code_challenge = BASE64URL(SHA-256(verifier))
2 Browser  /authorize?…&code_challenge=…&state=…
           user signs in on the auth server's own page
3 Redirect app receives ?code=…&state=…  (check state)
4 App      POST /token   code + code_verifier
5 Server   hashes verifier, compares → access + refresh
```

- اپ‌های native **کلاینت عمومی** (public client) هستند: نمی‌توانند client secret را مخفی نگه دارند
- PKCE کد را به همان نمونه از اپ که flow را شروع کرده گره می‌زند — کد دزدیده‌شده بدون verifier بی‌فایده است
- ورود در **مرورگر سیستم از طریق Custom Tabs** انجام می‌شود (کتابخانهٔ AppAuth همهٔ این‌ها را انجام می‌دهد) — هرگز در یک WebView داخلی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. PKCE در کد — و تله‌هایش

```
private val flags =
    Base64.URL_SAFE or Base64.NO_PADDING or Base64.NO_WRAP

fun newCodeVerifier(): String {
    val bytes = ByteArray(32)
    SecureRandom().nextBytes(bytes)
    return Base64.encodeToString(bytes, flags)
}

fun codeChallenge(verifier: String): String {
    val digest = MessageDigest.getInstance("SHA-256")
        .digest(verifier.toByteArray(Charsets.US_ASCII))
    return Base64.encodeToString(digest, flags)
}
```

تلهٔ مصاحبه

ورود از طریق WebView. اپ می‌تواند رمز عبور را بخواند، کاربر نمی‌تواند آدرس واقعی را ببیند، و گوگل ورود با حساب گوگل را در WebViewهای داخلی مسدود می‌کند. implicit flow (توکن در آدرس redirect) هم منسوخ شده — از کد به‌همراه PKCE استفاده کنید.

سؤال تکمیلی

«*کدام redirect URI؟*» — یک App Link تأییدشده با `https://` را ترجیح دهید. هر اپی می‌تواند همان custom scheme را ثبت کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. access token، refresh token و Authenticator

	access token	refresh token
طول عمر	چند دقیقه	چند روز تا چند ماه
فرستاده می‌شود به	همهٔ فراخوانی‌های API	فقط endpoint توکن
اگر دزدیده شود	پنجرهٔ آسیب کوتاه	جدی — چرخش و تشخیص استفادهٔ دوباره

```
class TokenAuthenticator(
    private val session: SessionManager,
) : Authenticator {
    override fun authenticate(
        route: Route?,
        response: Response,
    ): Request? {
        // Already retried once with a fresh token: give up
        if (response.priorResponse != null) return null
        val token = session.refreshBlocking() ?: return null
        return response.request.newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. قطعه‌ها چطور کنار هم قرار می‌گیرند

```
val client = OkHttpClient.Builder()
    .addInterceptor(AuthHeaderInterceptor(session)) // Day 11
    .authenticator(TokenAuthenticator(session))
    .build()
```

- **interceptor** توکن دسترسی را به همهٔ درخواست‌ها اضافه می‌کند
- وقتی پاسخی `401` باشد، OkHttp **Authenticator** را صدا می‌زند: یک درخواست جدید برگردانید تا تکرار شود، یا `null` تا تسلیم شود
- زنجیرهٔ `priorResponse` نشان می‌دهد چند بار تلاش کرده‌اید — هرگز بی‌پایان تکرار نکنید

تلهٔ مصاحبه

«*توکن‌ها را در EncryptedSharedPreferences ذخیره کن.*» — توصیهٔ پارسال. کتابخانهٔ security-crypto در Jetpack منسوخ شده؛ به‌جایش با کلیدی که در Keystore نگه داشته می‌شود رمز کنید (جزئیات در روز ۱۹). و تا جای ممکن access token را فقط در حافظه نگه دارید.

تلهٔ مصاحبه

refresh کردن با **همان** `OkHttpClient`: یک `401` از endpoint مربوط به refresh دوباره Authenticator را صدا می‌زند — بازگشت بی‌پایان. از یک client جدا بدون Authenticator استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

refresh هم‌زمان، کلید API و secretها، pagination و rate limit



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. پنج فراخوانی هم‌زمان 401 می‌گیرند

```
override fun authenticate(
    route: Route?,
    response: Response,
): Request? {
    if (response.priorResponse != null) return null
    val sent = response.request.header("Authorization")
    synchronized(lock) {
        val latest = session.accessToken()
        val token = when {
            // Another thread already refreshed: reuse its token
            latest != null && "Bearer $latest" != sent -> latest
            else -> session.refreshBlocking() ?: return null
        }
        return response.request.newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
    }
}
```

- هر بار فقط یک refresh؛ بقیه که منتظر بودند از توکن جدید استفاده می‌کنند
- `synchronized` این‌جا مشکلی ندارد: `authenticate()` روی تردهای پس‌زمینهٔ OkHttp اجرا می‌شود، هرگز روی ترد اصلی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. چرا مهم است

تلهٔ مصاحبه

پنج refresh موازی با refresh tokenهای **چرخشی**. اولی برنده می‌شود؛ چهار تای دیگر یک refresh token مصرف‌شده را می‌فرستند. بسیاری از سرورها این را دزدی تلقی می‌کنند و کل session را باطل می‌کنند — کاربر بی‌دلیل ظاهری از حسابش خارج می‌شود.

- اگر refresh برای همیشه شکست خورد، مثلاً با خطای `invalid_grant`، session را پاک کنید و یک state «خارج شده» منتشر کنید که UI آن را observe می‌کند — از ترد OkHttp ناوبری نکنید
- کمی قبل از `exp` پیشاپیش refresh کنید تا رفت‌وبرگشت اضافه حذف شود؛ Authenticator را به‌عنوان تور ایمنی نگه دارید

سؤال تکمیلی

«*با Ktor چطور؟*» — پلاگین `Auth` با `bearer { refreshTokens { … } }` چرخهٔ دریافت 401، refresh و تکرار را انجام می‌دهد. همان قواعد برقرار است: یک client جدا یا یک درخواست علامت‌گذاری‌شده برای فراخوانی refresh، و یک مسیر روشن برای «خارج شده».



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. کلید API و secretها: هیچ چیزی داخل APK مخفی نیست

- `BuildConfig`، `strings.xml`، فایل‌های native با پسوند `.so`، رشته‌های obfuscateشده — همه با ابزارهای معمولی در چند دقیقه استخراج می‌شوند
- پلاگین Secrets Gradle یا فایل `local.properties` کلیدها را از **git** دور نگه می‌دارد، نه از APK
- secretهای واقعی روی سرور شما زندگی می‌کنند: یک proxy در بک‌اند (BFF) API شخص ثالث را صدا می‌زند و اپ با توکن کاربر به بک‌اند **خودتان** احراز هویت می‌کند
- کلیدهایی که باید همراه اپ باشند (مثل SDK نقشه): در کنسول سرویس‌دهنده به package name و اثر انگشت گواهی امضا محدودشان کنید و سهمیه بگذارید
- هزینهٔ سوءاستفاده را با Play Integrity یا Firebase App Check بالا ببرید — نتیجه‌ای که **روی سرور** بررسی شود (روز ۲۰)

قاعدهٔ سرانگشتی

اگر لو رفتنش پول یا داده هزینه دارد، جایش داخل اپ نیست. شناسه بفرستید، نه secret.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تله‌های مربوط به secretها

تلهٔ مصاحبه

«*کلیدهای API را در build variantها نگه دار.*» — توصیهٔ پارسال. مقدارهای `buildConfigField` به‌صورت متن ساده داخل APK کامپایل می‌شوند. build variant محیط‌ها را از هم جدا می‌کند؛ چیزی را مخفی نمی‌کند.

تلهٔ مصاحبه

«*Play Integrity API، که قبلاً اسمش Play Protect بود.*» — این هم از پارسال است. Play Integrity جایگزین **SafetyNet Attestation API** شد. و نتیجه‌ای که فقط روی دستگاه چک شود را می‌شود با patch حذف کرد.

سؤال تکمیلی

«*پس اپ چطور یک API پولی شخص ثالث را صدا بزند؟*» — از طریق بک‌اند خودتان: اپ توکن کاربر را می‌فرستد، BFF کلید secret را اضافه می‌کند، سهمیهٔ هر کاربر را اعمال می‌کند و پاسخ‌ها را cache می‌کند. توکن کاربر لو رفته قابل باطل کردن است؛ کلید اصلی لو رفته نه.



اسماعیل مرادی
@esmaeeil-moradi700



سؤال ۱۰. pagination: offset یا cursor

	offset (`?page=3&size=20`)	cursor (`?after=abc&limit=20`)
تغییر داده حین اسکرول	آیتم تکراری یا جاافتاده	پایدار
پرش به صفحهٔ N	بله	خیر
صفحه‌های عمیق روی سرور	کند (رد کردن N ردیف)	سریع (جست‌وجو در index)

```
class FeedPagingSource(
    private val api: FeedApi,
) : PagingSource<String, PostDto>() {
    override suspend fun load(
        params: LoadParams<String>,
    ): LoadResult<String, PostDto> = try {
        val page = api.feed(params.key, params.loadSize)
        LoadResult.Page(page.items, null, page.nextCursor)
    } catch (e: IOException) {
        LoadResult.Error(e)
    } catch (e: HttpException) {
        LoadResult.Error(e)
    }

    override fun getRefreshKey(
        state: PagingState<String, PostDto>,
    ): String? = null
}
```



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۰. rate limit: کلاینت خوبی باشید

- پاسخ `429 Too Many Requests` معمولاً همراه `Retry-After` می‌آید؛ `503` هم ممکن است آن را داشته باشد
- سمت کلاینت: debounce ورودی جست‌وجو (روز ۵)، حذف فراخوانی‌های تکراری در جریان، دسته کردن درخواست‌های کوچک، cache
- اعمال واقعی محدودیت روی سرور است: برای هر کاربر، هر کلید، هر IP

```
fun retryAfterMillis(response: Response): Long? {
    val value = response.header("Retry-After") ?: return null
    value.toLongOrNull()?.let { return it * 1_000 }
    val date = response.headers.getDate("Retry-After")
        ?: return null
    val millis = date.time - System.currentTimeMillis()
    return millis.coerceAtLeast(0)
}
```

تلهٔ مصاحبه

«*pagination با offset برای یک فید زنده مشکلی ندارد.*» — پست‌های جدید همهٔ صفحه‌ها را جابه‌جا می‌کنند: آیتم تکراری در لیست، و `LazyColumn` با کلیدهای پایدار روی کلید تکراری کرش می‌کند. از cursor استفاده کنید (مبانی Paging 3 در روز ۶، و `RemoteMediator` در روز ۱۴).

تلهٔ مصاحبه

retry فوری روی `429` — یا همهٔ کلاینت‌ها بعد از یک تأخیر ثابت یکسان. فشار را بیشتر می‌کند. به `Retry-After` احترام بگذارید و jitter اضافه کنید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

فراتر از REST، اتصال‌های real-time، معماری امنیت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. فراتر از REST: انتخاب پروتکل

	شکل	کلاینت اندروید	مناسب برای	مراقب باشید
REST	منابع روی HTTP	Retrofit، Ktor	APIهای CRUD، HTTP caching	دادهٔ بیش از نیاز یا کمتر از نیاز
GraphQL	یک endpoint؛ کلاینت فیلدها را انتخاب می‌کند	Apollo Kotlin (KMP)	صفحه‌های زیادی که هر کدام بخشی از یک گراف را می‌خواهند	cache سمت کلاینت؛ خطا داخل پاسخ `200`
gRPC	Protobuf روی HTTP/2، stub تولیدشده	grpc-kotlin، Wire	APIهای داخلی، پرترافیک یا streaming	HTTP/2 سرتاسری لازم است؛ بررسی دستی سخت
WebSocket	دوطرفه و ماندگار	OkHttp، Ktor	چت، همکاری زنده	اتصال دوباره، مقیاس سرور، محدودیت‌های background
SSE	رویداد از سرور به کلاینت روی HTTP	okhttp-sse، Ktor	نتایج زنده، پیشرفت کار، فید	فقط یک‌طرفه

برای هر تعامل انتخاب کنید، نه برای کل شرکت

REST برای منابع، یک stream برای به‌روزرسانی زنده وقتی صفحه باز است، و push (همان FCM) برای background. بیشتر اپ‌های واقعی از هر سه استفاده می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. تله‌های هر کدام

تلهٔ مصاحبه

«*خطاهای GraphQL به شکل خطای HTTP می‌رسند.*» — خیلی وقت‌ها پاسخ `200 OK` می‌گیرید با یک آرایهٔ `errors`، شاید کنار `data` ناقص. هر دو را چک کنید، وگرنه شکست‌ها به شکل صفحهٔ خالی نمایش داده می‌شوند.

تلهٔ مصاحبه

«*gRPC سریع‌تر است، پس برای API عمومی موبایل از آن استفاده کن.*» — Protobuf باینری و HTTP/2 کمک می‌کنند، اما HTTP caching ساده و دیباگ خوانا را از دست می‌دهید، و هر proxy در مسیر باید از HTTP/2 پشتیبانی کند. آن را برای یک دلیل مشخص انتخاب کنید، نه فقط سرعت.

سؤال تکمیلی

«*برای شکل دادن پاسخ برای هر صفحه به GraphQL نیاز داریم؟*» — نه. یک backend-for-frontend (همان BFF) با endpointهای REST که برای هر صفحه شکل گرفته‌اند همین مشکل را با HTTP caching ساده حل می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. real-time: یک WebSocket به شکل Flow

```
fun OkHttpClient.messages(url: String) = callbackFlow<String> {
    val listener = object : WebSocketListener() {
        override fun onMessage(ws: WebSocket, text: String) {
            trySend(text)
        }
        override fun onFailure(
            ws: WebSocket,
            t: Throwable,
            response: Response?,
        ) {
            close(t)
        }
    }
    val request = Request.Builder().url(url).build()
    val socket = newWebSocket(request, listener)
    awaitClose { socket.close(1000, null) }
}
```

- `awaitClose` وقتی collector می‌رود socket را می‌بندد — مدیریت lifecycle به‌صورت رایگان
- تنظیم `pingInterval(…)` روی `OkHttpClient` ضربان (heartbeat) می‌فرستد و اتصال‌هایی را که شبکهٔ موبایل بی‌صدا قطع کرده تشخیص می‌دهد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. مثل یک سینیور دوباره وصل شوید

```
client.messages(url)
    .retryWhen { cause, attempt ->
        val transient = cause is IOException
        if (transient) delay(backoffWithJitter(attempt))
        transient
    }
```

- **ادامه دهید، نه فقط دوباره وصل شوید:** شناسهٔ آخرین رویداد یا شمارهٔ ترتیب را بفرستید و فاصلهٔ جاافتاده را از REST بگیرید
- فقط وقتی صفحه دیده می‌شود collect کنید (`WhileSubscribed` و `repeatOnLifecycle`)؛ در background به FCM تکیه کنید (روز ۱۶)

تلهٔ مصاحبه

اتصال دوبارهٔ فوری در یک حلقه. بعد از deploy سرور، یک میلیون کلاینت در همان ثانیه دوباره وصل می‌شوند — یک هجوم هم‌زمان (thundering herd) که سرور را دوباره پایین می‌آورد. backoff با jitter.

تلهٔ مصاحبه

«WebSocket را در یک سرویس background باز نگه دار تا پیام‌ها فوری برسند.» — باتری را خالی می‌کند، و محدودیت‌های background در هر صورت متوقفش می‌کنند. از یک پیام FCM با اولویت بالا برای بیدار کردن اپ و همگام‌سازی استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. معماری امنیت شبکه: از تهدید تا کنترل

تهدید	کنترل
شنود روی Wi-Fi عمومی	TLS همه‌جا؛ cleartext خاموش در Network Security Config
گواهی اشتباه صادرشده برای دامنهٔ شما	پایش CT؛ پین با پین پشتیبان، جایی که توجیه دارد
دزدیده شدن access token	عمر کوتاه، چرخش refresh، باطل کردن سمت سرور؛ توکن‌های وابسته به فرستنده (DPoP، mTLS)
کلاینت دست‌کاری‌شده یا اسکریپتی	attestation که روی سرور بررسی شود؛ مجوزدهی در هر فراخوانی
لو رفتن secret شخص ثالث	هیچ secretی داخل APK نیست — BFF نگهشان می‌دارد
scraping و brute force	محدودیت نرخ و سهمیه برای هر کاربر، کلید و IP
توکن و دادهٔ شخصی در لاگ‌ها	هدرهای redactشده؛ بدون لاگ body در release

تلهٔ مصاحبه

«*API ما امن است چون فقط اپ خودمان endpointها را می‌داند.*» — هر کسی با یک proxy در چند دقیقه فهرستشان را درمی‌آورد. هر endpoint باید هر درخواست را طوری مجوزدهی کند که انگار از یک اسکریپت آمده.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. کنترل‌ها کجا زندگی می‌کنند

```
App ── TLS + user token ──▶ BFF / API gateway
                            │ auth · rate limits · shaping
                            ├──▶ internal services (mTLS)
                            └──▶ third-party APIs (secrets)
```

- **BFF** پاسخ‌ها را برای هر صفحه شکل می‌دهد، secretهای شخص ثالث را نگه می‌دارد و محدودیت‌ها را در یک جا اعمال می‌کند
- **mTLS**: کلاینت هم گواهی ارائه می‌دهد — بین سرویس‌ها استاندارد است؛ روی دستگاه فقط وقتی که بتوانید کلیدهای کلاینت را توزیع و جایگزین کنید
- **OWASP MASVS-NETWORK** چک‌لیست خوبی برای بازبینی است: همهٔ ترافیک را امن کنید، فقط endpointهایی را پین کنید که در اختیار خودتان است

جواب سینیور در یک جمله

«شبکه خصمانه است و کلاینت هم: TLS برای کانال، توکن کوتاه‌عمر برای هویت، سرور برای همهٔ تصمیم‌ها — و فقط چیزی را پین کن که بتوانی نگهداری‌اش کنی.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۲ — خلاصهٔ یک‌صفحه‌ای

- TLS محرمانگی، یکپارچگی و احراز هویت سرور می‌دهد — نه هویت کلاینت.
- cleartext به‌طور پیش‌فرض مسدود است (از targetSdk 28)؛ به CA کاربر اعتماد نمی‌شود (از 24). آن را فقط در `debug-overrides` بگذارید.
- هرگز `TrustManager` یا `HostnameVerifier` «اعتماد به همه» منتشر نکنید.
- کلید عمومی را با پین پشتیبان و تاریخ انقضا پین کنید — یا اصلاً پین نکنید. نسخه‌های قدیمی پین را برای همیشه نگه می‌دارند.
- پین جلوی مهاجم شبکه را می‌گیرد، نه کسی که اپ را روی دستگاه خودش بررسی می‌کند.
- ورود در اپ native: کد مجوز با PKCE در Custom Tabs. بدون client secret، بدون WebView، بدون implicit flow.
- interceptor توکن را اضافه می‌کند؛ `Authenticator` پاسخ `401` را مدیریت می‌کند — هر بار یک refresh، client جدا، یک بار تکرار.
- هیچ چیزی داخل APK مخفی نیست: secret واقعی پشت بک‌اند می‌ماند؛ آنچه باید همراه اپ باشد را محدود و attest کنید.
- برای دادهٔ زنده pagination با cursor؛ به `429` و `Retry-After` با jitter احترام بگذارید.
- REST برای منابع، stream وقتی صفحه دیده می‌شود، FCM در background — و سرور همه چیز را اعمال می‌کند.

روز ۱۳ در راه است: لایهٔ داده، بخش ۱. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700